

Documentación Externa

Generación de código MIPS

Estudiantes

Geovanni González Aguilar

Escuela de Ingeniería en Computación

Curso

IC5701 - Compiladores e Intérpretes

Profesor

Allan Rodríguez Dávila

Grupo

60

II Semestre 2026

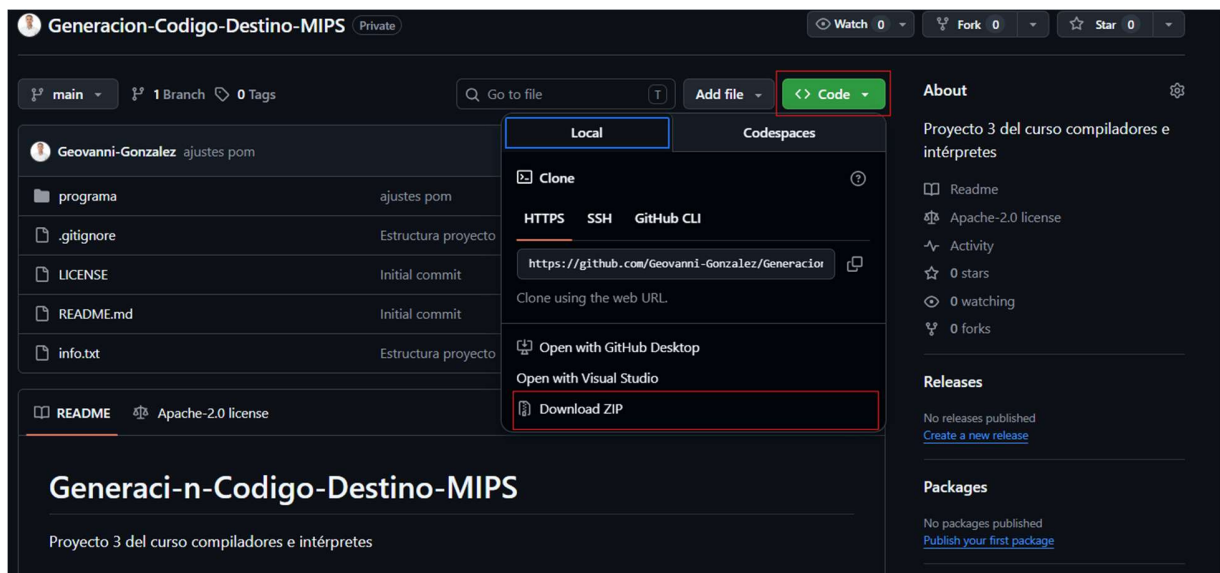
1. Manual de usuario:

1.1 Requisitos del sistema

Requisito	Versión mínima	Notas
Java (JDK)	17	Necesario para compilar y ejecutar el proyecto.
Apache Maven	3.6 o superior	Gestiona la construcción y las dependencias.
Simulador MIPS	MARS 4.5 / SPIM	Para ejecutar el código .asm generado.

1.2 Instalación y compilación

- 1.2.1 Se descarga y el archivo .zip desde el link de repositorio
Al descargarlo, lo descomprimos



1.2.2 Abre una nueva terminal y accede a la carpeta programa

Comando: `cd programa`

```
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS> cd programa
```

1.2.3 Compile y empaquete el proyecto con Maven

Comando: `mvn clean package`

```
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> mvn clean package
[INFO] Scanning for projects...
```

1.2.4 Ejecución del compilador

Formato: `java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar <archivo_fuente> [directorio_salida]`

- `<archivo_fuente>` (*obligatorio*): ruta del programa a compilar.
- `[directorio_salida]` (*opcional*): carpeta donde se escribirán los reportes y el código generado.

Ejemplo:

```
\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar test\04_aritmeticas.chip resultados
```

Resultado:

```
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\04_aritmeticas.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\resultados\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\resultados\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\resultados\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\resultados\resultado_sintactico.txt
Codigo intermedio: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\resultados\04_aritmeticas.ic
Codigo MIPS: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\resultados\04_aritmeticas.asm
```

1.2.5 Archivos de salida

En el directorio se generan los siguientes archivos:

Archivo	Contenido
tokens_report.txt	Lista de todos los tokens reconocidos, con su lexema, línea y columna.
tabla_simbolos.txt	Tabla de símbolos con identificadores, literales y palabras reservadas.
errores_report.txt	Resumen y detalle de los errores léxicos, sintácticos y semánticos encontrados.
resultado_sintactico.txt	Indica si el archivo fuente es aceptado por la gramática.

<nombre>.ic	Código intermedio de tres direcciones (solo si el programa es válido).
<nombre>.asm	Código destino en ensamblador MIPS (solo si el programa es válido).

2. Pruebas de funcionalidad

2.1 Caso de éxito

2.1.1 Paso 1 - Archivo fuente (caso_01.chip)

```

programa > test > caso_01.chip
1  empty ~ __main__<| |>
2  |:
3      int ~ x <- 10 + 5 * 2 !
4      cout <|x|> !
5  :|

```

Figura 1. Programa fuente del Caso 1.

2.1.2 Paso 2 – Compilación

Se ejecuta el compilador sobre el archivo fuente:

```
java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "caso_01.chip"
```

Salida en consola:

```

Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\caso_01.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\caso_01.ic
Codigo MIPS: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\caso_01.asm

```

Figura 2. El compilador acepta el programa y genera los archivos .ic y .asm.

2.1.3 Paso 3- Código intermedio generado

```

programa > salida > caso_01.ic
1 // Codigo intermedio
2 // Fecha: 2026-06-27
3 // Archivo fuente: caso_01.chip
4 // Integrantes: Geovanni Gonzalez
5
6 begin_function __main__
7 declare int x
8 x = 20
9 _t2 = load x
10 print _t2
11 end_function __main__
12

```

Figura 3. Código intermedio de tres direcciones. La expresión $10 + 5 * 2$ fue evaluada en tiempo de compilación y reducida a $x = 20$.

2.1.4 Paso 4 – Código MIPS

```

programa > salida > caso_01.asm
1 .data
2 d_main_x: .word 0
3 d_main_t2: .word 0
4
5 .text
6 .globl main
7
8 main:
9     li $t0, 20
10    sw $t0, d_main_x
11    sw $t0, d_main_t2
12    move $a0, $t0
13    li $v0, 1
14    syscall
15 main_fin:
16    li $v0, 10
17    syscall
18

```

Figura 4. Código ensamblador MIPS. Se reserva memoria para las variables en .data, se carga el valor 20, se imprime con la llamada al sistema y el programa finaliza.

2.1.5 Paso 5 - Ejecución en QtSpim



Figura 5. Ejecución del código MIPS en QtSpim; la consola muestra el resultado 20, confirmando la correcta traducción del programa fuente.

2.2 Caso de error

2.2.1 Paso 1- Archivo fuente (24_llamadas_funciones.chip)

```
programa > test > 24_llamadas_funciones.chip
1  int ~ sumar<|int ~ a, int ~ b|>
2  |:
3      return ~ a + b !
4  :|
5
6  empty ~ __main__<| |>
7  |:
8      int ~ correcto <- sumar<|1, 2|> !
9      int ~ noDeclarada <- desconocida<|1|> !
10     int ~ malCantidad <- sumar<|1|> !
11     int ~ malTipo <- sumar<|1, true|> !
12 :|
13
```

Figura 6. Programa fuente del Caso 2. La primera llamada es válida; las tres siguientes contienen errores semánticos intencionales.

2.2.2 Paso 2 – Compilación

Se ejecuta el compilador sobre el archivo fuente:

```
java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar " 24_llamadas_funciones.chip"
```

Salida en consola:

```
Error semantico [línea 9, col 1]: funcion 'desconocida' no declarada. Declare la funcion antes de invocarla y verifique la escritura de su nombre
Error semantico [línea 10, col 1]: cantidad de argumentos incorrecta: se esperaban 2 argumentos y se encontraron 1
Error semantico [línea 11, col 1]: argumento 2 incompatible: se esperaba tipo int y se encontro tipo bool
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\24_llamadas_funciones.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.
```

Figura 7. El compilador rechaza el programa e informa que el código intermedio y el MIPS no se generan debido a los errores de análisis.

2.2.3 Paso 3 – Reporte de errores

Al detectar los errores no genera los archivos .ic y .asm

```
programa > salida > errores_report.txt
1 RESUMEN: 3 error(es) encontrado(s) [lexicos: 0, sintacticos: 0, semanticos: 3]
2
3 ERRORES LEXICOS (0)
4 Sin errores lexicos.
5
6 ERRORES SINTACTICOS (0)
7 Sin errores sintacticos.
8
9 ERRORES SEMANTICOS (3)
10 1. Error semantico [línea 9, col 1]: funcion 'desconocida' no declarada. Declare la funcion antes de invocarla y verifique la escritura de su nombre
11 2. Error semantico [línea 10, col 1]: cantidad de argumentos incorrecta: se esperaban 2 argumentos y se encontraron 1
12 3. Error semantico [línea 11, col 1]: argumento 2 incompatible: se esperaba tipo int y se encontro tipo bool
```

Figura 8. Reporte de errores. El compilador clasifica los problemas por categoría e indica la línea de cada uno. Se detectan los tres errores semánticos : función no declarada, cantidad de argumentos incorrecta y tipo de argumento incompatible, mientras que la llamada correcta de la línea 8 no genera ningún error

3. Descripción del problema

Este proyecto corresponde a la etapa de generación de código, es decir, la construcción de la parte final del compilador. Toma como punto de partida el árbol de sintaxis abstracta producido por las fases anteriores y lo transforma en código ejecutable para la arquitectura MIPS.

El problema consiste en diseñar e implementar la fase de generación de código de un compilador, de modo que, a partir de un programa fuente sintáctica y semánticamente válido, se produzca un programa equivalente en ensamblador MIPS que pueda ejecutarse en un simulador estándar (QtSpim) y reproduzca el comportamiento esperado del programa original.

Para lograrlo es necesario resolver varios subproblemas:

- Representación intermedia: traducir las construcciones del lenguaje (declaraciones, expresiones, asignaciones, control de flujo, funciones y entrada/salida) a una representación intermedia de tres direcciones, independiente de la máquina destino, que sirva de puente entre el análisis y la generación final.
- Administración de recursos de la máquina: asignar registros y memoria de datos, respetando las convenciones de la arquitectura MIPS (banco de registros temporales, registros de punto flotante, pila para llamadas a funciones, etc.).
- Traducción a código destino: convertir cada instrucción intermedia en una o más instrucciones MIPS correctas, incluyendo el manejo de tipos (enteros y flotantes), operaciones aritméticas, lógicas y relacionales, estructuras de control y llamadas a funciones con paso de parámetros y retorno.
- Manejo de errores: garantizar que el código solo se genere cuando el programa es válido, y que cualquier error (léxico, sintáctico o semántico) se reporte de forma clara, sin producir código incorrecto.
- Generación de evidencia: producir los reportes que documentan el resultado del proceso (tokens, tabla de símbolos, errores y resultado del análisis), además de los archivos de código intermedio y de código destino.

3.1 Objetivos

3.1.1 Objetivo general

Implementar la fase de generación de código de un compilador para el lenguaje CHIP, capaz de traducir un programa fuente válido a código intermedio de tres direcciones y, a partir de este, a código ensamblador MIPS ejecutable en un simulador.

3.1.2 Objetivos específicos

- Construir una representación intermedia de tres direcciones a partir del árbol de sintaxis abstracta.
- Diseñar un esquema de administración de registros y de reserva de memoria de datos acorde a la arquitectura MIPS.
- Traducir a MIPS todas las construcciones del lenguaje: declaraciones, expresiones aritméticas, lógicas y relacionales, asignaciones, estructuras de control (if/else, do-while, switch), funciones y operaciones de entrada y salida.
- Soportar los tipos de dato del lenguaje, incluyendo operaciones con enteros y con punto flotante.
- Aplicar optimizaciones sobre el código generado (plegado de constantes y eliminación de redundancias de memoria) para producir un código más eficiente.

- Asegurar que el código solo se genere para programas válidos y reportar los errores detectados de manera estructurada.
- Validar el funcionamiento del compilador mediante un conjunto de casos de prueba y la ejecución del código resultante en el simulador QtSpim.

3.2 Alcance

El compilador desarrollado abarca el proceso completo, desde el archivo fuente hasta el código destino:

- Análisis léxico, sintáctico y semántico del lenguaje CHIP.
- Generación de código intermedio de tres direcciones.
- Optimización del código intermedio y del código MIPS.
- Generación de código ensamblador MIPS ejecutable en QtSpim.
- Emisión de reportes de tokens, tabla de símbolos, errores y resultado del análisis.

3.3 Restricciones

- El programa fuente debe contener exactamente un procedimiento principal `__main__`, de tipo `empty` y sin parámetros.
- El código intermedio y el código MIPS solo se generan cuando el programa no presenta errores léxicos, sintácticos ni semánticos.
- El código destino está dirigido a la arquitectura MIPS y utiliza las llamadas al sistema (*syscalls*) propias de los simuladores QtSpim para la entrada y salida.

4. Librerías usadas

El proyecto está desarrollado en Java 17 y se construye con Apache Maven, que gestiona la compilación, las dependencias y el empaquetado. Las librerías empleadas se agrupan según su función dentro del compilador: generación de los analizadores, ejecución, pruebas y construcción del proyecto.

4.1 Generación de analizadores

Constituyen el núcleo del compilador, ya que generan automáticamente el analizador léxico y el sintáctico a partir de especificaciones de alto nivel.

Librería	Versión	Función en el proyecto
JFlex	1.9.1	Generador de analizadores léxicos. A partir del archivo de especificación <code>lexico.flex</code> produce la clase <code>MiLexer</code> , encargada de reconocer los tokens del lenguaje.

Java CUP (generador)	11a	Generador de analizadores sintácticos <i>LALR</i> . A partir de la gramática <code>sintactico.cup</code> produce las clases <code>Parser</code> y <code>sym</code> , que realizan el análisis sintáctico y, dirigido por <code>syntaxis</code> , el semántico.
-----------------------------	-----	--

Estas dos herramientas se distribuyen como archivos `.jar` incluidos en el repositorio (`programa/src/lib/`), de modo que el proyecto se construye sin descargas adicionales:

- `jflex-full-1.9.1.jar`
- `java-cup-11a.jar`

4.2 Dependencias en tiempo de ejecución

Dependencia	Versión	Función en el proyecto
java-cup-runtime (com.github.vbmacher)	11b-20160615	Biblioteca de soporte en tiempo de ejecución que utiliza el analizador sintáctico generado por CUP (clases como <code>Symbol</code> y la maquinaria del <i>parser</i>). Se incluye también de forma local como <code>java-cup-11b-runtime.jar</code> .

4.3 Pruebas

Dependencia	Versión	Función en el proyecto
JUnit Jupiter (JUnit 5) (org.junit.jupiter)	5.10.2	Marco de pruebas unitarias. Permite ejecutar la batería de pruebas automatizadas que validan la compilación de programas válidos y el rechazo de programas inválidos.

5. Análisis de resultados:

5.1 Objetivos alcanzados

5.1.1 Generación de código intermedio

El compilador transforma correctamente el árbol de sintaxis abstracta en código de tres direcciones, una representación intermedia independiente de la máquina. Esto se evidencia en los archivos `.ic` generados, donde se observan instrucciones como `declare`, asignaciones, operaciones binarias, etiquetas (`_L0`), saltos (`if_false ... goto`) y llamadas a función. Esta capa intermedia permitió separar el análisis de la generación final y facilitó la optimización.

5.1.2 Administración de registros y memoria

Se implementó un administrador del banco de registros temporales (\$t0–\$t5) que entrega y libera registros durante la traducción, junto con un esquema de reserva de memoria en el segmento .data para variables y temporales. Los registros de apoyo (enteros y de punto flotante) quedaron centralizados y documentados, evitando colisiones con el banco administrado.

5.1.3 Cobertura de las construcciones del lenguaje

Todas las construcciones del lenguaje CHIP se traducen a MIPS: declaraciones de variables y arreglos, expresiones aritméticas, lógicas y relacionales, asignaciones, estructuras de control (if/else, do-while, switch), funciones con paso de parámetros y retorno, y operaciones de entrada/salida. Esta cobertura se valida con el conjunto de casos de prueba descrito en la sección de Pruebas de funcionalidad.

5.1.4 Soporte de tipos y punto flotante

El compilador maneja tanto enteros como números de punto flotante, generando las instrucciones y registros adecuados para cada tipo (l.s/s.s, registros \$f, conversiones y *syscalls* de impresión diferenciadas). Esto amplía el alcance más allá de la aritmética entera básica. Como resultado, el código MIPS generado es notablemente más corto y eficiente que la traducción directa.

5.1.5 Manejo de errores y robustez.

El compilador genera el código destino únicamente cuando el programa no presenta errores. Ante un programa inválido, reporta los errores clasificados por categoría (léxicos, sintácticos y semánticos) con su ubicación, y omite la generación de código. El Caso 2 evidencia la detección simultánea de tres errores semánticos sin interrumpirse en el primero, demostrando recuperación y ausencia de errores en cascada.

5.1.6 Validación y ejecución

La correcta operación del compilador se respalda con 30 pruebas automatizadas (programas válidos e inválidos) que se ejecutan exitosamente, complementadas con un flujo de integración continua que las ejecuta de forma automática. Además, el código MIPS generado fue ejecutado en el simulador QtSpim, produciendo los resultados esperados.

Detalle de los algoritmos más importantes del proyecto

Esta sección describe los algoritmos centrales de la fase de generación de código. Se presentan en el orden en que intervienen en el proceso de compilación: primero la traducción del árbol a código

intermedio, luego su optimización, después la traducción a MIPS con sus algoritmos de apoyo, y finalmente la optimización del código destino.

1. Generación de código intermedio dirigida por sintaxis

Propósito: transformar el árbol de sintaxis abstracta (AST) en código de tres direcciones, una secuencia lineal de instrucciones con a lo sumo tres operandos.

Idea del algoritmo: se recorre el AST en *post-orden*. Cada expresión se traduce a una función `generarExpresion` que devuelve el operando (un literal, una variable o un temporal) donde queda su valor; las subexpresiones se evalúan primero y sus resultados se combinan en un nuevo temporal. Las sentencias y estructuras de control se traducen generando etiquetas y saltos.

`generarExpresion(e)`:

si e es literal: retornar `valor_textual(e)`

si e es identificador: $t \leftarrow \text{nuevoTemporal}()$

 emitir(`LOAD, t, e.nombre`)

 retornar t

si e es binaria: $\text{izq} \leftarrow \text{generarExpresion}(e.\text{izquierda})$

$\text{der} \leftarrow \text{generarExpresion}(e.\text{derecha})$

$t \leftarrow \text{nuevoTemporal}()$

 emitir(`e.operador, t, izq, der`)

 retornar t

si e es llamada: para cada argumento: emitir(`PARAM, generarExpresion(arg)`)

$t \leftarrow \text{nuevoTemporal}()$

 emitir(`CALL, t, e.nombre, cantidadArgs`)

 retornar t

Los temporales se nombran `_t0, _t1, ...` mediante un contador, y las etiquetas `_L0, _L1, ....`

Ejemplo (`int ~ x <- 10 + 5 * 2`):

`_t0 = 5 * 2`

`_t1 = 10 + _t0`

`x = _t1`

1.1 Traducción de estructuras de control

El control de flujo se traduce con la instrucción `if_false c goto L` (saltar si la condición es falsa), `goto L` y etiquetas:

Condicional if/else:

cond $\leftarrow$ generarExpresion(condicion)

emitir(IF_FALSE, cond, Lelse)

generarBloque(thenBloque)

emitir(GOTO, Lfin)

emitir(LABEL, Lelse)

generarBloque(elseBloque)

emitir(LABEL, Lfin)

Ciclo do-while (el cuerpo se ejecuta al menos una vez):

emitir(LABEL, Linicio)

generarBloque(cuerpo)

cond $\leftarrow$ generarExpresion(condicion)

emitir(IF_FALSE, cond, Lfin)

emitir(GOTO, Linicio)

emitir(LABEL, Lfin)

2. Plegado y propagación de constantes con eliminación de temporales muertos

Propósito: evaluar en tiempo de compilación las operaciones aritméticas entre constantes y eliminar los temporales que dejan de usarse, produciendo un código intermedio más corto.

Idea del algoritmo: una pasada hacia adelante mantiene una tabla constantes que asocia cada temporal con su valor literal (los temporales son de asignación única, por lo que un valor constante es válido en todo el programa). En cada instrucción se sustituyen los operandos que sean temporales constantes; si una operación aritmética queda con dos literales enteros, se evalúa y se reemplaza por una asignación directa. Una segunda pasada elimina las definiciones de temporales que ya no se leen.

plegarConstantes(codigo):

constantes $\leftarrow$ {}

para cada instrucción i en codigo:

op1 $\leftarrow$ sustituir(i.op1, constantes) // si es temporal constante, usar su valor

op2 $\leftarrow$ sustituir(i.op2, constantes)

si i es aritmética entera y op1, op2 son literales y resultado ≥ 0 :

valor $\leftarrow$ evaluar(i.op, op1, op2)

emitir(ASIG, i.resultado, valor)

si i.resultado es temporal: constantes[i.resultado] $\leftarrow$ valor

si no:

emitir(i con operandos sustituidos)

eliminarMuertos(codigo):

leidos $\leftarrow$ temporales que aparecen como lectura en alguna instrucción

descartar toda definición de temporal cuyo resultado $\notin$ leidos

Restricciones de seguridad: solo se pliegan enteros no negativos (para no emitir literales que el traductor MIPS no representaría) y se evita la división por cero; nunca se eliminan instrucciones con efectos (llamadas, lecturas, saltos).

Ejemplo (caso 04_aritmeticas):

Antes: Después:

_t0 = 5 * 2 x = 20

_t1 = 10 + _t0 y = 30

x = _t1 z = 2

...

3. Administración de registros temporales

Propósito: repartir y reutilizar el banco de registros temporales \$t0–\$t5 durante la traducción a MIPS.

Idea del algoritmo: se mantiene una pila (deque) de registros disponibles y un conjunto de ocupados. Al pedir un registro se extrae uno disponible; al liberarlo, se devuelve a la pila para reutilizarlo en la siguiente operación. Como el código de tres direcciones consume sus operandos de inmediato, el banco de seis registros es suficiente.

obtenerRegistro():

si no hay disponibles: lanzar error controlado

r $\leftarrow$ sacar de disponibles

marcar r como ocupado

retornar r

liberarRegistro(r):

si r estaba ocupado:

marcar r como libre

devolver r a disponibles

4. Cálculo de direcciones de arreglos bidimensionales

Propósito: calcular la dirección de memoria de una celda $m[i][j]$ de un arreglo bidimensional almacenado por filas (*row-major*).

Idea del algoritmo: un arreglo de C columnas almacena el elemento $[i][j]$ en el desplazamiento lineal $(i \cdot C + j)$. Como cada palabra ocupa 4 bytes, la dirección final es:

$$\text{direccion}(m[i][j]) = \text{base}(m) + (i \cdot C + j) \cdot 4$$

En MIPS se traduce cargando los índices, multiplicando la fila por el número de columnas, sumando la columna, escalando por 4 (`sll ... 2`) y sumando la dirección base:

cargar i, cargar j

li cols, C

mul i, i, cols

add i, i, j

sll i, i, 2 # × 4 bytes por palabra

la dir, base(m)

add dir, dir, i

5. Fusión de comparación y salto condicional

Propósito: optimizar el patrón "calcular un booleano y luego saltar si es falso", evitando materializar el valor booleano en un registro.

Idea del algoritmo: al traducir, se examina cada par de instrucciones. Si una comparación entera es seguida inmediatamente por un `if_false` que usa su resultado, ambas se fusionan en una sola instrucción de salto condicional con la comparación negada (porque `if_false` salta cuando la condición es *falsa*). La correspondencia de negación es:

Comparación (intermedio)	Salto MIPS (negado)
<code>==</code> (igual)	<code>bne</code>
<code>!=</code> (distinto)	<code>beq</code>
<code><</code> (menor)	<code>bge</code>
<code><=</code> (menor o igual)	<code>bgt</code>

Comparación (intermedio)

Salto MIPS (negado)

> (mayor)

ble

>= (mayor o igual)

blt

Si no se cumple el patrón (o los operandos son flotantes), se traduce de la forma general: se evalúa la condición y se emite `beq cond, $zero, destino`.

Ejemplo (condición $i \leq 5$ del ciclo del Caso 3): el par `_t6 = i <= 5 + if_false _t6 goto _L1` se traduce en una sola instrucción:

```
bgt $t0, $t1, _ic_L1    # salir del ciclo si i > 5
```

6. Optimización *peephole* de redundancias de memoria

Propósito: eliminar los viajes redundantes a memoria del código MIPS emitido.

Idea del algoritmo: se recorre el listado de instrucciones MIPS observando pares adyacentes. Si una instrucción guarda un registro en una etiqueta de datos (`sw/s.s`) y la siguiente vuelve a cargar esa misma etiqueta (`lw/l.s`), el valor ya está en el registro de origen, por lo que:

- si el registro destino es el mismo, la carga se elimina;
- si es distinto, la carga se reemplaza por un `move/mov.s`.

para cada par (`store`, `load`) adyacente:

si `store` es `sw/s.s` a etiqueta directa

y `load` es `lw/l.s` de la misma etiqueta:

conservar `store`

si `reg(store) = reg(load)`: eliminar `load`

si no: reemplazar `load` por `"move reg(load), reg(store)"`

Para garantizar la correctitud, solo se aplica a etiquetas de datos directas (no a formas con desplazamiento como `0($t7)`), evitando cualquier supuesto de *aliasing*; el `store` siempre se conserva.

Ejemplo:

Antes:

Después:

```
sw $t2, d_main_t10      sw $t2, d_main_t10
```

```
lw $t0, d_main_t10      move $t0, $t2
```

7. Despacho de la traducción a MIPS

Propósito: convertir cada instrucción de código intermedio en su equivalente MIPS.

Idea del algoritmo: el generador actúa como despachador: recorre el código intermedio y, según la operación de cada instrucción, delega en el traductor especializado correspondiente (memoria, operaciones aritméticas/lógicas, control de flujo, funciones o entrada/salida). Antes de traducir, intenta la fusión de comparación y salto (algoritmo 5); si no aplica, traduce la instrucción de forma individual.

traducir(codigo):

para cada instrucción i (con índice):

si i e i+1 forman un par comparación + if_false fusionable:

traducirSaltoComparacion(i, i+1); saltar i+1

si no:

despachar(i) según i.op → traductor especializado

Esta organización modular mantiene el generador principal simple y concentra la lógica de cada familia de operaciones en su propia clase.

Bitácora (autogenerada en git, commit por usuario incluyendo comentario).

Link Github:

[Geovanni-Gonzalez/Generacion-Codigo-Destino-MIPS: Proyecto 3 del curso compiladores e intérpretes](#)